CHAPTER 8

Outbound HTTP Requests with Req

Applications rarely exist in isolation. They use APIs (Application Programming Interfaces) to interact with other applications—whether internal or external—to achieve their value. Applications interact with each other in many different ways, but the most common form of interaction is an HTTP-based API. By the end of this chapter, you'll have what you need to make your own HTTP requests from Elixir.

On the surface, making HTTP requests seems like an easy problem to solve. We pick an HTTP request library, integrate it into our application, and then move on with our day. But the problem is a bit more complex than that. (Don't worry, the end result will be straightforward.)

HTTP clients in Elixir have been continually evolving and growing over the years. When compared to Ruby, you'll see a lot of different Elixir HTTP clients. This means that several options can fill our needs, and it can be a little bit confusing as to which library we should use. Libraries also sit at different levels of abstraction, so they have different purposes. But don't worry, we'll use a client that's great for all applications.

In this chapter, we'll look at several of the different HTTP clients that are out there. We'll settle on Req as our first-choice client, and you'll use it to implement an API in our SMS app. And along the way, we'll cover additional topics such as application configuration and secrets.

Let's look at the different HTTP clients that are available in the Elixir ecosystem.

Decide on Which HTTP Client to Use

HTTP clients in Elixir have a long history. Even before Elixir, there were Erlang clients that are still used to this day in Elixir apps. Knowing the whole history doesn't provide you with much benefit, but it will be helpful to understand the different abstraction levels that clients can sit on top of.